

Projet de programmation encadré - LYSET13

Git : introduction et préparer son environnement de travail

Pierre Magistry pierre.magistry@inalco.fr

Yoann Dupont yoann.dupont@sorbonne-nouvelle.fr

2026–2027

Université Sorbonne-Nouvelle
INALCO
Université Paris-Nanterre

Les bases



Git...

- ... est un **système de gestion de versions** ou SGV (en anglais *Version Control Software* ou *VCS*).

Git...

- ... est un **système de gestion de versions** ou SGV (en anglais *Version Control Software* ou *VCS*).
- ... permet de gérer les **modifications** effectuées sur un dossier données de manière **décentralisée**.

Git...

- ... est un **système de gestion de versions** ou SGV (en anglais *Version Control Software* ou *VCS*).
- ... permet de gérer les **modifications** effectuées sur un dossier données de manière **décentralisée**.
- ... ne versionne pas des fichiers, mais des ensembles ordonnés de modifications.

Git...

- ... est un **système de gestion de versions** ou SGV (en anglais *Version Control Software* ou *VCS*).
- ... permet de gérer les **modifications** effectuées sur un dossier données de manière **décentralisée**.
- ... ne versionne pas des fichiers, mais des ensembles ordonnés de modifications.

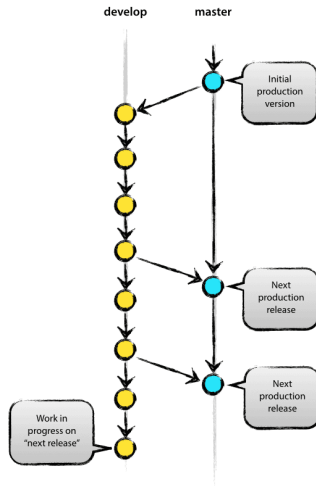


Figure 1: Une vision simplifiée d'un dépôt.
À gauche, vos changements locaux, à droite, l'état de votre dépôt distant.

Git et Github sont deux choses différentes :

- Git est un outil créé par Linus Torvald (créateur du noyau Linux)
 - programme qui sera à installer sur votre ordinateur
- GitHub est un service web construit autour de Git créé par l'entreprise Github, Inc. (rachetée par Microsoft en 2018)
 - Offre un espace de stockage en ligne pour les dépôts Git
 - Mais aussi d'autres fonctionnalités absentes de Git seul

Préparation de l'environnement

Installer git

Git est installé par défaut sur Kubuntu, rien à faire !

Si ce n'est pas le cas, deux choix :

1. aller sur Discover et cherchez *git*.
2. ouvrir un terminal¹, puis `sudo apt install git`.

¹ctrl+alt+t sous *ubuntu, launchpad → terminal sous macOS

Installer git

Git est installé par défaut sur Kubuntu, rien à faire !

Si ce n'est pas le cas, deux choix :

1. aller sur Discover et cherchez *git*.
2. ouvrir un terminal¹, puis `sudo apt install git`.

Pour vérifier que git est bien installé, on teste cette commande sur un terminal :

`git`

¹ctrl+alt+t sous *ubuntu, launchpad → terminal sous macOS

Créer un compte Github

Allez sur <https://github.com> et cliquez sur *sign up* en haut à droite/gauche (dépend de la taille de l'écran).

Créer un compte Github

Allez sur <https://github.com> et cliquez sur *sign up* en haut à droite/gauche (dépend de la taille de l'écran).

Il est possible que GitHub vous demande de configurer l'authentification à deux facteurs (2FA). Cela ne sera pas forcément nécessaire, au moins pour aujourd'hui.

Créer un compte Github

Allez sur <https://github.com> et cliquez sur *sign up* en haut à droite/gauche (dépend de la taille de l'écran).

Il est possible que GitHub vous demande de configurer l'authentification à deux facteurs (2FA). Cela ne sera pas forcément nécessaire, au moins pour aujourd'hui.

Sinon, suivez la doc : <https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/configuring-two-factor-authentication>

Prévoyez un smartphone qui scanne les QR code pour y installez une application qui gèrera les mots de passe de type TOTP (*time-based one time password*).

Parmi les apps citées par GitHub, privilégiez 1password, Authy, etc. (évitez l'app Microsoft, GAFAM tout ça).

GitHub est une plateforme collaborative sur laquelle circule notamment du code.

- Besoin de sécuriser le contenu des dépôts (usurpation, vandalisme, etc.)
 - Sécuriser des informations privées (chiffrer)
 - Authentification de l'auteur·ice d'un message (signer)
- Une solution prévue dans github consiste à ajouter une clé de sécurité

Aussi appelée Cryptographie à Clé Publique (*Public Key Cryptography*).

Sans rentrer dans les détails techniques, il faut comprendre la finalité :

- Vous avez deux clés : une **publique** (que tout le monde peut voir) et une **privée** (qui doit n'être accessible qu'à vous).
- cela permet de chiffrer et signer ce que vous envoyez vers / recevez de Github.

GitHub devient de plus en plus strict avec la sécurité :

- Vous aurez sans doute besoin d'une clé pour pousser vos changements.
- Type de chiffage recommandé : Ed25519

GitHub devient de plus en plus strict avec la sécurité :

- Vous aurez sans doute besoin d'une clé pour pousser vos changements.
- Type de chiffage recommandé : Ed25519

Comment utiliser ces clés sur GitHub ?

1. Créer la paire clé publique/privée sur votre ordinateur
2. Donner la clé **publique** à GitHub

```
ssh-keygen -t ed25519
```

source : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

```
ssh-keygen -t ed25519
```

La commande vous demandera alors :

1. Où sauvegarder la clé (par défaut la clé privée dans `~/.ssh/id_ed25519` et la publique dans `~/.ssh/id_ed25519.pub`)
2. Un mot de passe pour la clé (qu'il faudra entrer à chaque fois)
 - 2.1 On peut ne rien donner, mais pas recommandé

source : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

Ajouter la clé à votre agent SSH

Vérifier que votre agent SSH tourne :

```
$ eval "$(ssh-agent -s)"
```

source : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

Ajouter la clé à votre agent SSH

Vérifier que votre agent SSH tourne :

```
$ eval "$(ssh-agent -s)"
```

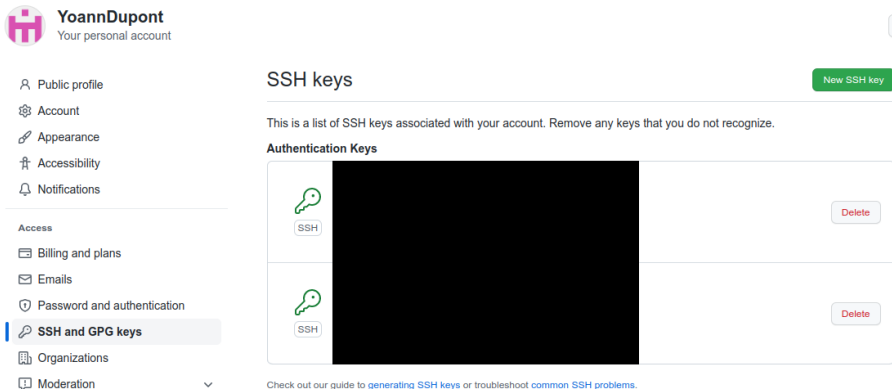
Affichera un message type "Agent pid XXXX". Si tel est le cas, on ajoute la clé à l'agent (en supposant que ~/.ssh/id_ed25519 est votre clé privée) :

```
ssh-add ~/.ssh/id_ed25519
```

source : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

Ajouter la clé à GitHub

Finalement, on ajoute la clé **publique** (termine par .pub) à GitHub via les *settings*.



The screenshot shows the GitHub settings interface for the user 'YoannDupont'. On the left is a sidebar with navigation links: Public profile, Account, Appearance, Accessibility, Notifications, Access, Billing and plans, Emails, Password and authentication, SSH and GPG keys (highlighted), Organizations, and Moderation. The main content area is titled 'SSH keys' and includes a 'New SSH key' button. Below the title is a description: 'This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.' Under the heading 'Authentication Keys', there is a table with two rows, each representing an SSH key. The key names are redacted with a black box. Each row has a green key icon, an 'SSH' label, and a 'Delete' button. At the bottom, there is a link to a guide on generating SSH keys and troubleshooting common problems.

YoannDupont
Your personal account

Public profile
Account
Appearance
Accessibility
Notifications

Access
Billing and plans
Emails
Password and authentication
SSH and GPG keys
Organizations
Moderation

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication Keys

 SSH	[Redacted]	Delete
 SSH	[Redacted]	Delete

Check out our guide to [generating SSH keys](#) or troubleshoot [common SSH problems](#).

source : <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account>

Les commandes git

Git fournit un ensemble de commandes qui permettent de gérer les changements.

La syntaxe générale prend la forme suivante :

```
git <sous-commande> [-options...] [arguments...]
```

Git fournit un ensemble de commandes qui permettent de gérer les changements.

La syntaxe générale prend la forme suivante :

```
git <sous-commande> [-options...] [arguments...]
```

Les commandes git sont paramétrables, on ne traitera ici que des cas de base.

En accord avec la philosophie Unix, les commandes git sont également très précises et font le moins de choses possibles.

1ère commande avec exemple : git clone

Permet de créer une copie d'un dépôt existant sur sa machine.

Cette copie peut de suivre et de faire suivre les changements apportés au dépôt. On a donc un **clone** du dépôt distant.

```
git clone [-options...] <URL>
```

Où <URL> est le lien vers un dossier git (généralement distant).

1ère commande avec exemple : git clone

Permet de créer une copie d'un dépôt existant sur sa machine.

Cette copie peut de suivre et de faire suivre les changements apportés au dépôt. On a donc un **clone** du dépôt distant.

```
git clone [-options...] <URL>
```

Où <URL> est le lien vers un dossier git (généralement distant).

Lier un dépôt de travail à son compte GitHub

En supposant que USER soit votre identifiant GitHub et EMAIL votre email :

```
git config user.name "USERNAME"
```

```
git config user.email "EMAIL"
```

On peut mettre l'option `--global` pour ne pas faire ces deux lignes à chaque fois.

On dit bien un clone, pas un fork !
Si vous forcez, vous ne verrez aucune mise à jour.

Petit test - récupérer les slides du cours

Créez-vous un dossier pluralité quelque part **PAS SUR LE BUREAU**.

Allez dedans et tapez la commande suivante :

```
git clone git@github.com:YoannDupont/PPE1-2627.git
```

Inutile de lier vos identifiants à ce dépôt, vous ne pourrez pas pousser.

Permet de mettre-à-jour votre branche vers la bonne version (par défaut : la dernière version en ligne).

En termes git, on **tire** les changements du dépôt distant vers notre répertoire local.

```
git pull
```

Permet de mettre-à-jour les métadonnées du dépôt sur votre répertoire local.

Le terme **fetch** fait sans doute référence à **play fetch** (jouer à la ramener balle).

```
git fetch
```

Permet de mettre-à-jour les métadonnées du dépôt sur votre répertoire local.

Le terme **fetch** fait sans doute référence à **play fetch** (jouer à la ramener balle).

```
git fetch
```

`git fetch` a quelques avantages par rapport à `pull` :

- Fonctionne toujours (ajoute simplement plus de métadonnées).
- Ne modifie pas les fichiers du dépôt.
- Permet d'anticiper les conflits (changements incompatibles).

Permet d'indiquer des fichiers dont on veut suivre les modifications avant validation.

En terminologie git, on **ajoute au suivi** des modifications faites sur des fichiers. On appelle cette étape la mise-en-place (*staging*).

```
git add <FILE...>
```

Où <FILE...> est un ou plusieurs fichiers dont on souhaite suivre les modifications

Permet d'indiquer des fichiers dont on veut suivre les modifications avant validation.

En terminologie git, on **ajoute au suivi** des modifications faites sur des fichiers. On appelle cette étape la mise-en-place (*staging*).

```
git add <FILE...>
```

Où <FILE...> est un ou plusieurs fichiers dont on souhaite suivre les modifications

Attention :

git add suit les modifications **passées, mais pas futures**. Si vous modifiez un fichiers après un add, ces nouveaux changements ne seront pas suivis.

Permet de **retirer du suivi** (*remove*) un fichier. Cela équivaut à supprimer le fichier des futures versions.

```
git rm <FILE...>
```

Où <FILE...> est un ou plusieurs fichiers dont on souhaite supprimer du dépôt. Retirer un fichier via `git rm` met en place le changement automatiquement.

Permet de **retirer du suivi** (*remove*) un fichier. Cela équivaut à supprimer le fichier des futures versions.

```
git rm <FILE...>
```

Où <FILE...> est un ou plusieurs fichiers dont on souhaite supprimer du dépôt. Retirer un fichier via `git rm` met en place le changement automatiquement.

Attention :

- Un fichier retiré du dépôt peut être à nouveau ajouté par la suite.
- Un fichier retiré du dépôt ne disparaît pas totalement : il demeure accessible sur les versions précédentes.

Permet de valider les modifications des fichiers suivis.

Pourquoi *commit* ? De nouvelles modifications peuvent être faites après un commit, mais git **se tiendra** aux modifications validées.

```
git commit [-m message]
```

Où message est le message qui décrit les changements que vous faites.

Permet de valider les modifications des fichiers suivis.

Pourquoi *commit* ? De nouvelles modifications peuvent être faites après un commit, mais git **se tiendra** aux modifications validées.

```
git commit [-m message]
```

Où message est le message qui décrit les changements que vous faites.

Attention :

Si aucun message n'est donné, git ouvrira un éditeur de texte dans le terminal pour que vous puissiez écrire. Ça peut faire de mauvaises surprises.

Envoie les modifications mises en place (ou "commitées") vers le dépôt distant.

On dit, en git, qu'on **pousse** les modifications.

```
git push
```

Permet de voir les changements de votre dossier par rapport à la version du dépôt.

```
git status
```

Cela affichera les fichiers mis en place (*staged*) et les modifications non suivies.

Permet de voir l'ensemble des *commits* ayant été effectués sur le dépôt du plus récent au plus ancien.

```
git log
```

Log vous indiquera notamment les auteur·ices, dates et messages des commits. Il faut appuyer sur la touche "q" pour sortir du log.

Mettre à jour un dépôt depuis votre ordinateur

Le travail sur un dépôt git suit une certaine routine. Le scénario le plus simple :

- on répète plusieurs fois :
 - `git pull`
 - On fait des modifications...
 - `git add / git rm`
 - `git commit`
 - `git push`

Mettre à jour un dépôt depuis votre ordinateur

Le travail sur un dépôt git suit une certaine routine. Le scénario le plus simple :

- on répète plusieurs fois :
 - `git pull`
 - On fait des modifications...
 - `git add / git rm`
 - `git commit`
 - `git push`
- quand on a fini :
 - `git tag`

Mettre à jour un dépôt depuis votre ordinateur

Le travail sur un dépôt git suit une certaine routine. Le scénario le plus simple :

- on répète plusieurs fois :
 - `git pull`
 - On fait des modifications...
 - `git add / git rm`
 - `git commit`
 - `git push`
- quand on a fini :
 - `git tag`

`git tag ?`

GitHub : Les étiquettes

Quoi et pourquoi

Dans la longue chaîne des modifications, il peut être difficile de savoir si des commits sont plus intéressants que d'autres.

- Les étiquettes (*tag*) permettent de marquer un commit particulier.
- La valeur et la signification du tag dépend uniquement des mainteneurs.
- GitHub transforme cependant les tags en "*release*", ce qui est un parti pris.

source : <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

Quoi et pourquoi

Dans la longue chaîne des modifications, il peut être difficile de savoir si des commits sont plus intéressants que d'autres.

- Les étiquettes (*tag*) permettent de marquer un commit particulier.
- La valeur et la signification du tag dépend uniquement des mainteneurs.
- GitHub transforme cependant les tags en "*release*", ce qui est un parti pris.

Utilisation des tags dans ce cours :

Vous utiliserez les tags pour nous indiquer **uniquement** les versions finies de vos travaux pour que nous allions les vérifier.

source : <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

On peut ajouter un tag à un commit :

```
git tag [-a] [-m message] <tagname> [commit]
```

- tagname est le nom du tag (seul élément non-optionnel).
- [commit] indique le commit qu'on veut tagger (sinon, le commit courant).
- -a permet d'annoter un tag avec un message donné par -m.

source : <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

On peut ajouter un tag à un commit :

```
git tag [-a] [-m message] <tagname> [commit]
```

- tagname est le nom du tag (seul élément non-optionnel).
- [commit] indique le commit qu'on veut tagger (sinon, le commit courant).
- -a permet d'annoter un tag avec un message donné par -m.

Pousser un tag vers GitHub :

```
git push origin <tagname>
```

source : <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

On peut ajouter un tag à un commit :

```
git tag [-a] [-m message] <tagname> [commit]
```

- `tagname` est le nom du tag (seul élément non-optionnel).
- `[commit]` indique le commit qu'on veut tagger (sinon, le commit courant).
- `-a` permet d'annoter un tag avec un message donné par `-m`.

Pousser un tag vers GitHub :

```
git push origin <tagname>
```

à faire maintenant avec le `<tagname>` *seance1*.

source : <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

Dans les cas simples, on a tendance à suivre cette façon de faire :

- plusieurs modifications locales $\implies$ un *add*

Dans les cas simples, on a tendance à suivre cette façon de faire :

- plusieurs modifications locales $\implies$ un *add*
- 1 à plusieurs *add* $\implies$ 1 *commit*

Dans les cas simples, on a tendance à suivre cette façon de faire :

- plusieurs modifications locales $\implies$ un *add*
- 1 à plusieurs *add* $\implies$ 1 *commit*
- 1 à plusieurs *commits* $\implies$ 1 *push*

Dans les cas simples, on a tendance à suivre cette façon de faire :

- plusieurs modifications locales $\implies$ un *add*
- 1 à plusieurs *add* $\implies$ 1 *commit*
- 1 à plusieurs *commits* $\implies$ 1 *push*
- 1 à plusieurs *push* $\implies$ 1 *tag*

Dans les cas simples, on a tendance à suivre cette façon de faire :

- plusieurs modifications locales $\implies$ un *add*
- 1 à plusieurs *add* $\implies$ 1 *commit*
- 1 à plusieurs *commits* $\implies$ 1 *push*
- 1 à plusieurs *push* $\implies$ 1 *tag*

Soyez certain(e) de ce que vous faites quand vous créez un *tag*. On en veut, pour ce semestre, **1 par séance, pas plus**.

Et maintenant la pratique